Bill also changes the callers to use String#to_alphanumeric. For example, the test becomes as follows:

```
require 'test/unit'

class StringExtensionsTest < Test::Unit::TestCase
  def test_strip_non_alphanumeric_characters
    assert_equal '3 the Magic Number', '#3, the *Magic, Number*?'.to_alphanumeric
  end
end
```

To understand the previous trick, you need to know a thing or two about Ruby classes. Bill is only too happy to teach you....

Inside Class Definitions

In Ruby, there is no real distinction between code that defines a class and code of any other kind. You can put any code you want in a class definition:

```
3.times do
  class C
    puts "Hello"
  end
end
```

```
Hello
Hello
Hello
```

Ruby executed the code within the class just as it would execute any other code. Does that mean you defined three classes with the same name? The answer is no, as you can quickly find out yourself:

```
class D
  def x; 'x'; end
end

class D
  def y; 'y'; end
end

obj = D.new
obj.x        # => "x"
obj.y        # => "y"
```

When the previous code mentions class D for the first time, no class by that name exists yet. So, Ruby steps in and defines the class—and the x method. At the second mention, class D already exists, so Ruby doesn't need to define it. Instead, it reopens the existing class and defines a method named y there.